



Clasificador de Componentes Electrónicos en Tiempo Real

Agostini, Emiliano S.

Departamento de Ingeniería Electrónica,
Facultad Regional San Francisco, Córdoba, Argentina.

Cortesse, Agustín

Departamento de Ingeniería Electrónica,
Facultad Regional San Francisco, Córdoba, Argentina.

Índice

1. Resumen	3
2. Introducción	3
3. Herramientas utilizadas	4
4. Descripción del sistema	5
4.1. Preparación del conjunto de datos	6
4.1.1. Dataset	7
4.1.2. Recorte de imágenes	9
4.2. Entrenamiento del modelo	9
4.2.1. Parámetros utilizados	11
4.2.2. Modelo entrenado	11
4.3. Clasificación en tiempo real	13
5. Pruebas del sistema	14
6. Conclusiones	16

1. Resumen

El presente trabajo consiste en el desarrollo de un sistema de visión artificial capaz de identificar componentes electrónicos en tiempo real mediante el uso de una cámara web. Para ello se construyó un conjunto de imágenes correspondiente a cuatro clases: resistencias, LED, capacitores electrolíticos y capacitores cerámicos. Dichas imágenes fueron organizadas y divididas en conjuntos de entrenamiento y validación.

El modelo utilizado se basa en MobileNetV2, el cual fue adaptado mediante transferencia de aprendizaje para clasificar únicamente las clases del proyecto. El entrenamiento se realizó utilizando PyTorch, mientras que la captura de video y el procesamiento en tiempo real se implementaron con OpenCV. Una vez entrenado, el sistema es capaz de recibir imágenes en tiempo real, procesarlas y devolver la clase del componente detectado junto con su nivel de confianza.

Los resultados obtenidos muestran un desempeño satisfactorio en condiciones controladas, demostrando la viabilidad de utilizar técnicas de aprendizaje profundo para la identificación automática de componentes electrónicos.

2. Introducción

El presente trabajo tiene como objetivo desarrollar un sistema de visión artificial capaz de reconocer componentes electrónicos utilizando una cámara web y técnicas de inteligencia artificial.

Para ello se entrenó un modelo de clasificación de imágenes con cuatro tipos de componentes:

- Resistencias.
- LED.
- Capacitores electrolíticos.
- Capacitores cerámicos.

Una vez finalizado el entrenamiento, el modelo puede recibir imágenes provenientes de una cámara en tiempo real e identificar automáticamente el componente observado mostrando además el porcentaje de confianza de la predicción.

El desarrollo fue realizado completamente en Python utilizando la biblioteca PyTorch para el entrenamiento del modelo y OpenCV para la captura de video.

Como objetivos específicos tenemos:

- Construir un conjunto de imágenes para el entrenamiento.
- Entrenar un modelo de clasificación utilizando PyTorch.
- Implementar un sistema de reconocimiento en tiempo real mediante una cámara web.
- Evaluar el desempeño del modelo obtenido.

3. Herramientas utilizadas

Durante el desarrollo del proyecto se utilizaron distintas herramientas de software, cada una destinada a una etapa específica del sistema. Todo el desarrollo se realizó sobre el sistema operativo Windows, utilizando Visual Studio Code como entorno de desarrollo integrado (IDE) y un entorno virtual de Python (`venv`) para aislar las dependencias del proyecto.

Herramienta	Versión	Función
Python	3.14	Lenguaje utilizado para desarrollar todo el proyecto.
PyTorch	2.12.0 (CPU)	Entrenamiento del modelo mediante transferencia de aprendizaje, cálculo de gradientes y optimización de los pesos de la red.
Torchvision	—	Provee la arquitectura MobileNetV2 con pesos preentrenados en ImageNet, junto con las transformaciones de preprocesamiento (<i>resize</i> , <i>normalización</i> , <i>data augmentation</i>).
OpenCV	4.13.0	Captura de video desde la cámara web, lectura de fotografías, recorte interactivo mediante mouse, y dibujo del overlay de resultados sobre cada fotograma.
PIL (Pillow)	—	Conversión del formato de color de las imágenes (BGR a RGB) y aplicación de las transformaciones del modelo antes de cada inferencia.
NumPy	—	Operaciones numéricas sobre los vectores de probabilidad devueltos por el modelo, utilizadas en el suavizado temporal de las predicciones.

Cuadro 1: Herramientas utilizadas en el proyecto.

La elección de PyTorch como framework de aprendizaje profundo, por sobre otras alternativas como TensorFlow, respondió a su sintaxis flexible y a la amplia disponibilidad de modelos preentrenados a través del módulo `torchvision`, lo cual simplificó considerablemente la implementación del transfer learning sobre MobileNetV2.

4. Descripción del sistema

El sistema desarrollado está dividido en tres etapas principales:

1. Preparación del conjunto de imágenes.
2. Entrenamiento del modelo.
3. Clasificación en tiempo real.

Archivo	Función
dataset.py	Organiza las imágenes y crea los conjuntos de entrenamiento y validación.
train.py	Entrena el modelo y guarda el archivo resultante.
inference.py	Utiliza el modelo entrenado para reconocer componentes mediante la cámara web.

Cuadro 2: Archivos principales del proyecto.

La separación en tres archivos independientes no es una decisión estética sino técnica. En primer lugar, permite reutilizar el modelo sin repetir el entrenamiento: una vez guardado el archivo **.pth**, `inference.py` puede ejecutarse ilimitadas veces sin recalcul nada. En segundo lugar, si se quiere agregar datos al dataset, basta con volver a ejecutar `dataset.py` y `train.py` sin tocar `inference.py`. En tercer lugar, si se mejora la arquitectura de red, `train.py` puede actualizarse de forma aislada.

Adicionalmente, el proyecto fue versionado mediante **Git**, lo cual permitió registrar la evolución del código y las distintas iteraciones realizadas durante el desarrollo. A partir de estas tres etapas conceptuales, el funcionamiento completo del sistema puede resumirse mediante el siguiente procedimiento operativo:

1. Capturar imágenes de cada componente electrónico.

2. Organizar las imágenes en carpetas según la clase.
3. Ejecutar el archivo `dataset.py`.
4. Entrenar el modelo mediante `train.py`.
5. Guardar el modelo entrenado.
6. Ejecutar `inference.py`.
7. Clasificar componentes en tiempo real utilizando la cámara web.

4.1. Preparación del conjunto de datos

Para entrenar el modelo fue necesario construir un conjunto de imágenes correspondiente a cada una de las clases que se desean reconocer. La totalidad de las fotografías fue capturada utilizando la cámara de un teléfono celular, lo cual permitió obtener imágenes de mayor resolución y nitidez en comparación con una webcam convencional, aportando mayor cantidad de detalle al momento de extraer características visuales durante el entrenamiento.

Para la captura del conjunto de imágenes se utilizó como soporte una base de fondo blanco, con el objetivo de mantener condiciones de iluminación y contraste homogéneas entre las distintas tomas. Esta homogeneidad resulta relevante desde el punto de vista del aprendizaje, ya que reduce la variabilidad introducida por factores ajenos al componente en sí (como cambios de fondo o de iluminación entre fotografías), permitiendo que el modelo concentre su capacidad de aprendizaje en las características propias de cada clase –forma, color y geometría– en lugar de aprender patrones espurios asociados al entorno de captura. Asimismo, el alto contraste entre el componente oscuro y el fondo claro resultó beneficioso para una etapa posterior de recorte de las imágenes, en la cual se buscó maximizar la proporción del componente dentro del encuadre.

Por otro lado, la cantidad y diversidad de las muestras capturadas constituye un factor determinante en la capacidad de generalización del modelo. Un conjunto de datos reducido o poco variado tiende a producir sobreajuste (*overfitting*), es decir, un modelo que memoriza las particularidades de las imágenes de entrenamiento en lugar de aprender los rasgos generales que distinguen a cada clase, lo cual deriva en un desempeño deficiente ante situaciones no vistas durante el entrenamiento, como las que se presentan en la clasificación en tiempo real. Para mitigar este riesgo, cada componente fue fotografiado desde tres vistas diferentes: vista lateral, vista a 45 y vista superior. Para cada una de estas vistas, el componente fue rotado sobre su propio eje en un rango de entre 10 y 15 posiciones distintas.

Esta variabilidad angular busca que el modelo aprenda a reconocer el componente independientemente de su orientación relativa respecto a la cámara, condición que no puede controlarse durante el uso real del sistema, en el cual el usuario coloca el componente frente a la cámara en una posición arbitraria. Las imágenes fueron organizadas inicialmente dentro de la carpeta:

```
data/raw/
```

Dentro de esta carpeta se creó un directorio para cada componente.

```
|-- data/
|-- raw/
    |-- resistencia/
    |-- led/
    |-- capacitor_electrolitico/
    |-- capacitor_ceramico/
```

4.1.1. Dataset

Una vez finalizada la construcción del conjunto de imágenes, se ejecutó el script `dataset.py`, encargado de dividir el conjunto total de imágenes en los subconjuntos de entrenamiento y validación, en una proporción de 80 % y 20 % respectivamente.

Esta separación constituye una práctica estándar en el desarrollo de modelos de aprendizaje automático, y resulta fundamental para evaluar correctamente el desempeño del sistema. El conjunto de entrenamiento es utilizado por el modelo para ajustar sus parámetros internos: la red ve las imágenes, comete errores, corrige sus pesos, y repite este proceso muchas veces hasta que logra clasificarlas correctamente. Sin embargo, dado que el modelo ha sido expuesto repetidamente a esas imágenes, no es posible utilizarlas para medir su rendimiento real, ya que podría haber simplemente memorizado sus características particulares en lugar de haber aprendido a generalizar. Es por ello que se reserva un conjunto de validación, compuesto por imágenes que el modelo nunca observa durante el entrenamiento. Al finalizar cada época, el modelo es evaluado sobre este conjunto sin modificar sus pesos. La precisión obtenida sobre el conjunto de validación constituye entonces una estimación honesta de la capacidad del modelo para clasificar imágenes nuevas, evitando que se reporte un desempeño artificialmente alto producto de la memorización de los datos de entrenamiento.

Para realizar esta división, el script mezcla aleatoriamente las imágenes de cada clase utilizando una semilla fija (`random.seed(42)`). El uso de una semilla fija garantiza que la división resulte reproducible: si el script se ejecuta nuevamente sobre el mismo conjunto de imágenes, el resultado de la separación será idéntico, lo cual resulta conveniente al momento de comparar distintas configuraciones de entrenamiento bajo las mismas condiciones de datos. La división se realiza de manera independiente para cada clase, lo cual asegura que la proporción 80/20 se mantenga de forma equilibrada entre todas las categorías, evitando que alguna clase quede subrepresentada en el conjunto de validación.

La Tabla 3 resume la cantidad final de imágenes obtenidas por clase luego de la división.

Clase	Entrenamiento	Validación
Resistencia	226	57
LED	213	54
Capacitor electrolítico	272	68
Capacitor cerámico	192	49
Total	903	228

Cuadro 3: Distribución de imágenes por clase luego de la división en conjuntos de entrenamiento y validación.

Luego de la ejecución del script se obtiene la siguiente estructura.

```

|--processed/

  |-- train/
    |-- resistencia/
    |-- led/
    |-- capacitor_electrolitico/
    |-- capacitor_ceramico/

  |-- val/
    |-- resistencia/
    |-- led/
    |-- capacitor_electrolitico/
    |-- capacitor_ceramico/
  
```

Esta organización en subcarpetas por clase es requerida por la clase `ImageFolder` de `torchvision`, que durante el entrenamiento interpreta automáticamente cada subdirectorio como una etiqueta distinta, evitando la necesidad de construir manualmente un archivo de anotaciones.

4.1.2. Recorte de imágenes

Durante una primera etapa de pruebas se observó que parte de las fotografías capturadas presentaban al componente ocupando una porción reducida del encuadre, con un margen considerable de fondo. Esta condición dificulta el aprendizaje del modelo, ya que al redimensionar la imagen a 224×224 píxeles el componente queda representado por muy pocos píxeles, perdiendo gran parte de la información visual relevante.

Para corregir este inconveniente se desarrolló una herramienta de recorte interactivo (`manual_crop.py`), que permite seleccionar manualmente, mediante el mouse, la región de interés en cada fotografía. El usuario dibuja un rectángulo alrededor del componente y la herramienta guarda únicamente esa región en una carpeta separada, preservando las imágenes originales sin modificar.

Esta etapa permitió mejorar significativamente la proporción del componente dentro del encuadre, lo cual se reflejó en una mayor estabilidad y confianza de las predicciones durante la clasificación en tiempo real.

4.2. Entrenamiento del modelo

Una vez preparado el conjunto de datos, se procedió al entrenamiento del modelo utilizando PyTorch, una biblioteca de Python desarrollada por Meta que provee todas las herramientas necesarias para construir y entrenar redes neuronales. PyTorch se encarga de ejecutar las operaciones matemáticas sobre los datos, calcular automáticamente los gradientes durante el entrenamiento y gestionar la comunicación con el hardware disponible, ya sea CPU o GPU, sin que el programador deba intervenir en esos procesos de bajo nivel. Para este proyecto se eligió la arquitectura MobileNetV2, ya que ofrece un buen equilibrio entre precisión y velocidad de procesamiento, permitiendo realizar clasificaciones en tiempo real sin requerir hardware de alto rendimiento, a diferencia de arquitecturas más profundas como ResNet o VGG, cuyo costo computacional dificultaría su ejecución fluida sobre una computadora convencional sin unidad de procesamiento gráfico (GPU).

En lugar de entrenar una red desde cero, se utilizaron los pesos preentrenados del modelo sobre el conjunto de datos ImageNet, el cual contiene más de un millón de imágenes pertenecientes a mil categorías distintas. Esta técnica, conocida como transferencia de aprendizaje (*transfer learning*), parte del supuesto de que las capas convolucionales iniciales de una red ya entrenada han aprendido a extraer características visuales genéricas –bordes, texturas, formas y patrones de color– que resultan útiles independientemente del dominio específico de la tarea final.

Para aprovechar este conocimiento previo, los parámetros de todas las capas convolucionales de MobileNetV2 fueron congelados (`requires_grad = False`), de modo que no se actualizan durante el entrenamiento. Únicamente se reemplazó la última capa completamente conectada (*classifier*) por una nueva capa lineal con cuatro salidas, una por cada clase del proyecto, siendo esta la única porción de la red cuyos pesos se entrenan desde cero. Esta estrategia reduce drásticamente la cantidad de parámetros a optimizar y, en consecuencia, la cantidad de imágenes y tiempo de cómputo necesarios para obtener un modelo funcional, en comparación con un entrenamiento completo de la red.

Desde el punto de vista de su arquitectura, MobileNetV2 corresponde a una **red neuronal convolucional** (*Convolutional Neural Network*, CNN), una familia de modelos especializada en el procesamiento de imágenes mediante la aplicación sucesiva de filtros convolucionales que extraen características visuales en distintos niveles de abstracción: las primeras capas detectan rasgos simples como bordes y texturas, mientras que las capas más profundas combinan estos rasgos para reconocer formas y estructuras más complejas.

La particularidad de MobileNetV2 frente a otras CNN reside en el uso de *convoluciones separables en profundidad* (*depthwise separable convolutions*), una técnica que descompone una convolución estándar en dos operaciones más livianas: una convolución *depthwise*, que procesa cada canal de la imagen de forma independiente, seguida de una convolución *pointwise*, que combina los resultados de todos los canales mediante filtros de 1×1 . Esta descomposición reduce drásticamente la cantidad de operaciones aritméticas necesarias en comparación con una convolución convencional, lo cual explica por qué esta arquitectura resulta apropiada para ejecutarse sobre una CPU convencional sin necesidad de una unidad de procesamiento gráfico, condición señalada como restricción de diseño en los objetivos del proyecto.

Adicionalmente, MobileNetV2 introduce el bloque denominado *inverted residual with linear bottleneck*, que expande temporalmente la cantidad de canales antes de aplicar la convolución *depthwise* y los comprime nuevamente al finalizar dicho bloque. Esta estrategia permite que la red mantenga una buena capacidad de representación interna sin incrementar significativamente la cantidad de parámetros entrenables. De esta manera, la eficiencia computacional propia de la arquitectura MobileNetV2 y la reutilización de conocimiento mediante transfer learning resultan complementarias: la primera permite ejecutar el modelo en tiempo real sobre hardware limitado, mientras que la segunda permite obtener un modelo preciso a partir de un conjunto de datos comparativamente pequeño, como el utilizado en este proyecto.

Durante el entrenamiento se evaluó el desempeño del modelo utilizando el conjunto de validación al finalizar cada época, entendiendo por época una pasada completa del modelo sobre la totalidad de las imágenes de entrenamiento. El modelo correspondiente a la época con mejor precisión de validación fue guardado automáticamente, descartando las épocas posteriores en caso de que su desempeño no superara al mejor registrado previamente.

4.2.1. Parámetros utilizados

Parámetro	Valor
Modelo	MobileNetV2
Cantidad de clases	4
Épocas	20
Batch Size	16
Learning Rate	0.001
Tamaño de imagen	224×224 píxeles
Optimizador	Adam
Función de pérdida	CrossEntropyLoss

Cuadro 4: Parámetros utilizados durante el entrenamiento.

La cantidad de épocas (20) se determinó de forma empírica, observando que la precisión sobre el conjunto de validación se estabilizaba en valores superiores al 98 % a partir de las primeras épocas, sin evidenciar mejoras significativas adicionales. El tamaño de lote (*batch size*) de 16 imágenes se eligió considerando que el entrenamiento se ejecuta sobre CPU, donde lotes de mayor tamaño incrementan el tiempo de cómputo por iteración sin aportar beneficios relevantes dado el tamaño del conjunto de datos disponible. El tamaño de imagen de 224×224 píxeles corresponde al tamaño de entrada requerido por la arquitectura MobileNetV2 en su configuración estándar. Como optimizador se utilizó **Adam**, ampliamente utilizado en tareas de *fine-tuning* por su capacidad de adaptar la tasa de aprendizaje de forma individual para cada parámetro, favoreciendo una convergencia más estable. Finalmente, al tratarse de un problema de clasificación multiclase, se utilizó la función de pérdida **CrossEntropyLoss**, estándar para este tipo de tareas.

4.2.2. Modelo entrenado

Al finalizar el entrenamiento se genera un archivo denominado:



`best_model.pth`

Este archivo contiene toda la información necesaria para realizar posteriormente la clasificación de imágenes sin volver a entrenar la red neuronal. De esta forma, el entrenamiento solo debe realizarse nuevamente cuando se agregan nuevas imágenes o se desea mejorar el modelo.

Una vez finalizado el entrenamiento se obtuvo un modelo capaz de reconocer los cuatro componentes electrónicos considerados en este trabajo. Para evaluar el desempeño se utilizó el conjunto de validación, compuesto por imágenes que no fueron utilizadas durante el entrenamiento. Durante cada época se registró la evolución de la precisión (*Accuracy*) y de la función de pérdida (*Loss*), permitiendo observar el aprendizaje del modelo. La Figura 1 muestra dicha evolución a lo largo de las 20 épocas de entrenamiento.

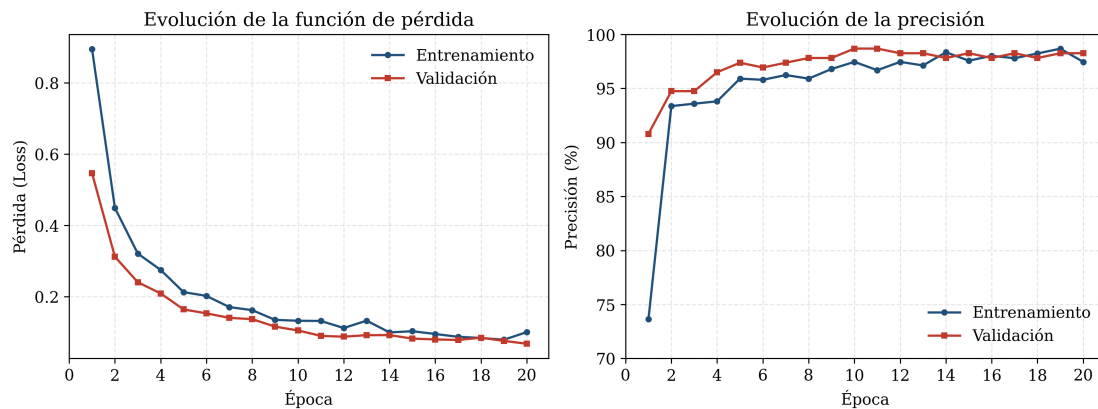


Figura 1: Evolución de la función de pérdida (izquierda) y de la precisión (derecha) sobre los conjuntos de entrenamiento y validación, a lo largo de las 20 épocas.

Como puede observarse, ambas curvas de pérdida descienden de forma sostenida durante las primeras épocas y se estabilizan a partir aproximadamente de la época 10, sin evidenciar un incremento de la pérdida de validación respecto de la de entrenamiento, lo cual indica que el modelo no presenta sobreajuste (*overfitting*) sobre el conjunto de datos disponible. De manera consistente, la precisión de validación se mantiene en todo momento igual o superior a la precisión de entrenamiento, alcanzando un máximo de 98,68 % en la época 10, valor que fue seleccionado como modelo final dado que `train.py` conserva automáticamente la época con mejor desempeño sobre el conjunto de validación.

4.3. Clasificación en tiempo real

El archivo `inference.py` utiliza el modelo previamente entrenado para identificar componentes electrónicos mediante una cámara web.

Cada imagen capturada pasa por una serie de etapas antes de obtener la predicción.

1. Captura de la imagen desde la cámara.
2. Conversión al formato requerido por el modelo.
3. Redimensionamiento a 224×224 píxeles.
4. Clasificación mediante MobileNetV2.
5. Suavizado temporal de la predicción.
6. Visualización del resultado en pantalla.

La Figura 2 muestra el funcionamiento general del sistema.



Figura 2: Flujo general de clasificación en tiempo real.

En primer lugar, OpenCV captura cada fotograma proveniente de la cámara en formato BGR (azul, verde, rojo), por lo cual es necesario convertirlo al formato RGB esperado por las bibliotecas PIL y `torchvision`. Posteriormente, la imagen es redimensionada a 224×224 píxeles y normalizada utilizando los mismos parámetros estadísticos empleados durante el entrenamiento, de manera que el modelo reciba las imágenes en las mismas condiciones sobre las cuales fue entrenado.

Una vez preprocesada, la imagen es evaluada por el modelo, el cual devuelve un vector de probabilidades mediante la función `softmax`, asignando a cada una de las cuatro clases un valor de probabilidad cuya suma total es igual a 1.

A diferencia de un esquema de clasificación frame a frame, en el cual la predicción podría oscilar de forma considerable ante variaciones momentáneas de enfoque, iluminación o posición del componente, se implementó un mecanismo de **suavizado temporal**. Para ello, el vector de probabilidades de cada fotograma se almacena en una estructura de tipo `deque` de tamaño fijo, conservando únicamente los últimos 12 fotogramas procesados. La predicción mostrada en pantalla no corresponde a

la clasificación de un único fotograma, sino al promedio de las probabilidades acumuladas en dicho buffer, seleccionando la clase con mayor probabilidad promedio. De esta forma, un fotograma puntual afectado por desenfoque de movimiento o un cambio brusco de iluminación queda compensado por los fotogramas restantes, reduciendo significativamente la oscilación de la predicción observada durante las primeras pruebas del sistema.

Finalmente, si la probabilidad de la clase ganadora no supera un umbral mínimo de confianza, establecido en un 60 %, el sistema no informa una clasificación específica, mostrando en su lugar un símbolo de interrogación. Esta decisión de diseño prioriza evitar una clasificación incorrecta antes que forzar una respuesta cuando el modelo no posee suficiente certeza.

5. Pruebas del sistema

Una vez entrenado el modelo se realizaron distintas pruebas utilizando una cámara web. En cada prueba se colocó un componente frente a la cámara y se verificó la respuesta del sistema. Durante la ejecución del programa se muestra la imagen proveniente de la cámara junto con la información correspondiente a la predicción realizada por el modelo.

En pantalla se visualizan:

- Nombre del componente detectado.
- Porcentaje de confianza.
- Imagen en tiempo real capturada por la cámara.

Cuando el modelo no alcanza un nivel mínimo de confianza, se evita mostrar una clasificación incorrecta, reduciendo así posibles errores durante el uso del sistema.

La Figura 3 muestra un ejemplo de funcionamiento.



Figura 3: Reconocimiento de un componente mediante la cámara web.

Durante las pruebas iniciales del sistema se identificaron una serie de inconvenientes que motivaron ajustes tanto en el conjunto de datos como en el código de inferencia. A continuación se describen los problemas detectados y las soluciones implementadas para cada caso.

Componente alejado del encuadre. En una primera instancia se observó que, al fotografiar el componente a una distancia considerable de la cámara, este ocupaba una porción muy reducida de la imagen capturada. Dado que el modelo redimensiona cada fotografía a 224×224 píxeles antes de procesarla, esta condición provocaba que el componente quedara representado por muy pocos píxeles, perdiendo gran parte de la información visual necesaria para una clasificación correcta. Como solución, se desarrolló la herramienta de recorte interactivo descrita en la Sección 4.1.2, mediante la cual se ajustó manualmente el encuadre de las fotografías del conjunto de datos para maximizar la proporción del componente dentro de cada imagen.

Confianza de predicción baja. Como consecuencia directa del problema anterior, las primeras pruebas mostraban una confianza de predicción considerablemente baja, incluso para componentes correctamente identificados. Tras aplicar el recorte de imágenes mencionado, se observó una mejora notable en los niveles de confianza reportados por el sistema durante la clasificación en tiempo real.

Oscilación de la predicción en tiempo real. Finalmente, se observó que la clase informada por el sistema podía variar de un fotograma a otro aun cuando el componente permanecía inmóvil frente a la cámara, particularmente en presencia de pequeños movimientos, variaciones de enfoque o cambios momentáneos de iluminación. Para mitigar este comportamiento se incorporó el mecanismo de

suavizado temporal, el cual promedia las probabilidades de los últimos 12 fotogramas antes de determinar la clase a mostrar en pantalla, logrando una predicción considerablemente más estable.

6. Conclusiones

En este trabajo se desarrolló integralmente un sistema de visión artificial basado en aprendizaje profundo para la clasificación automática de componentes electrónicos. El proyecto permitió recorrer todas las etapas necesarias para construir un sistema de este tipo, desde la preparación del conjunto de datos hasta la implementación de una aplicación funcional. La elección de la arquitectura MobileNetV2 fue clave, ya que permitió obtener un modelo liviano y de alto desempeño, logrando ejecutar la clasificación en tiempo real mediante una computadora convencional y una cámara web. Como resultado, se consolidó una herramienta destacada por su código modular, su interfaz intuitiva y su capacidad de operar fluidamente en el momento.

Si bien los resultados obtenidos fueron altamente satisfactorios, el sistema presenta algunas limitaciones inherentes a su etapa actual de desarrollo. Por el momento, el reconocimiento se restringe únicamente a los cuatro componentes utilizados durante el entrenamiento inicial. Asimismo, la precisión del modelo es sensible al entorno físico: el rendimiento puede disminuir ante variaciones drásticas de iluminación o cuando los componentes se encuentran muy alejados o parcialmente ocultos. Dado que el desempeño del sistema depende directamente de la calidad y el volumen del *dataset*, estas restricciones pueden mitigarse en el futuro aumentando el tamaño del conjunto de imágenes y ejecutando un nuevo entrenamiento.

De cara al futuro, la flexibilidad y modularidad del diseño dejan el camino preparado para diversas mejoras. Se propone escalar el proyecto incorporando nuevas familias de componentes electrónicos mediante bases de datos más extensas, así como diseñar una interfaz gráfica más robusta que pueda mostrar información técnica adicional sobre cada elemento detectado. Finalmente, existe un gran potencial para optimizar este modelo y desplegarlo en dispositivos embebidos, como una Raspberry Pi. En definitiva, los resultados demuestran que este tipo de herramientas son perfectamente aplicables a tareas de automatización e identificación visual, constituyendo una base sólida y escalable para futuros desarrollos orientados a aplicaciones industriales.

Referencias

Agostini, E. S. y Cortesse, A. (2026). *Clasificador de Componentes Electrónicos en Tiempo Real*. Repositorio del proyecto final – Visión Artificial, UTN FRSF. Recuperado de <https://github.com/emilianoagostini14/TP-ClasificadordComponentes>

OpenCV Team. (2015). *OpenCV: Open Source Computer Vision Library*. Recuperado de <https://opencv.org>

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... y Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Recuperado de <https://pytorch.org>

PyTorch Contributors. (2016). *torchvision: PyTorch Vision Library*. Recuperado de <https://github.com/pytorch/vision>

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., ... y Fei-Fei, L. (2015). *ImageNet Large Scale Visual Recognition Challenge*. Recuperado de <https://www.image-net.org>

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. y Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 4510–4520.